

1. Python Example: Point Estimation

```
import numpy as np

data = np.array([10,12,9,13,15,11,14,8])

# Point estimates
mean_est = np.mean(data)
var_est = np.var(data, ddof=1) # unbiased variance

print("Estimated Mean:", mean_est)
print("Estimated Variance:", var_est)
```

2. Python Example: Maximum Likelihood Estimation (Normal μ , σ)

```
import numpy as np

data = np.array([12, 13, 15, 11, 14, 10, 12])

# MLE for  $\mu$  and  $\sigma^2$ 
mu_mle = np.mean(data)
sigma2_mle = np.mean((data - mu_mle)**2)

print("MLE Mean:", mu_mle)
print("MLE Variance:", sigma2_mle)
```

3. Python Example: Confidence Interval (95%)

```
import numpy as np

from scipy.stats import t

data = np.array([10,12,9,13,15,11,14,8])

n = len(data)

mean = np.mean(data)

se = np.std(data, ddof=1) / np.sqrt(n)

t_critical = t.ppf(0.975, df=n-1)

lower = mean - t_critical * se

upper = mean + t_critical * se

print("95% Confidence Interval:", (lower, upper))
```

4. Python Program (T-Interval)

```
import numpy as np

from scipy.stats import t

data = np.array([12, 15, 14, 13, 16, 11, 12, 15, 14, 13])

n = len(data)

mean = np.mean(data)

sd = np.std(data, ddof=1)

se = sd / np.sqrt(n)

t_critical = t.ppf(0.975, df=n-1) # 95% CI

lower = mean - t_critical * se

upper = mean + t_critical * se

print("Mean:", mean)

print("95% Confidence Interval:", (lower, upper))
```

5. Python Program (Z-Interval)

```
import numpy as np

from scipy.stats import norm

data = np.array([12, 15, 14, 13, 16, 11, 12, 15, 14, 13])

sigma = 2 # known population SD
n = len(data)
mean = np.mean(data)

z = norm.ppf(0.975) # 95% CI

lower = mean - z * sigma/np.sqrt(n)
upper = mean + z * sigma/np.sqrt(n)

print("95% Confidence Interval:", (lower, upper))
```

6. Python Code to Plot 95% CI ($Z = \pm 1.96$)

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import norm

# X-axis values
x = np.linspace(-4, 4, 500)

# Standard normal PDF
```

```
y = norm.pdf(x)
```

```
# Critical values for 95% CI
```

```
z_critical = 1.96
```

```
plt.figure(figsize=(9, 5))
```

```
# Plot the normal curve
```

```
plt.plot(x, y, linewidth=2)
```

```
# Vertical lines at -1.96 and +1.96
```

```
plt.axvline(-z_critical, linestyle='--')
```

```
plt.axvline(z_critical, linestyle='--')
```

```
# Text labels
```

```
plt.text(-z_critical, 0.02, "-1.96", ha='right')
```

```
plt.text(z_critical, 0.02, "+1.96", ha='left')
```

```
# Tail probabilities 0.025 on each side
```

```
plt.text(-z_critical - 0.3, 0.06, "0.025")
```

```
plt.text(z_critical + 0.1, 0.06, "0.025")
```

```
# Center confidence level area
```

```
plt.text(0, 0.25, "0.95", ha='center', fontsize=12)
```

```
plt.title("95% Confidence Interval on Standard Normal Distribution")
```

```
plt.xlabel("Z-value")
plt.ylabel("Density")
```

```
plt.show()
```

7.95% CI for mean (σ unknown — t-interval)

```
import numpy as np
from scipy.stats import t

data = np.array([12,15,14,13,16,11,12,15])
n = len(data)
mean = data.mean()
s = data.std(ddof=1)
se = s/np.sqrt(n)
tcrit = t.ppf(0.975, df=n-1)
lower, upper = mean - tcrit*se, mean + tcrit*se

print("mean =", mean)
print("s =", s)
print("95% CI =", (lower, upper))
```

8. FULL PYTHON PROGRAM FOR ESTIMATION THEORY

```
import numpy as np
from scipy.stats import norm, t, chi2

# -----
```

1. POINT ESTIMATION

```
def point_estimates(data):
```

```
    xbar = np.mean(data)    # sample mean
```

```
    s = np.std(data, ddof=1) # sample standard deviation
```

```
    return xbar, s
```

2. CONFIDENCE INTERVAL — MEAN (σ KNOWN)

```
def ci_mean_sigma_known(xbar, sigma, n, confidence=0.95):
```

```
    alpha = 1 - confidence
```

```
    z = norm.ppf(1 - alpha/2)
```

```
    me = z * (sigma / np.sqrt(n))
```

```
    return xbar - me, xbar + me
```

3. CONFIDENCE INTERVAL — MEAN (σ UNKNOWN)

```
def ci_mean_sigma_unknown(data, confidence=0.95):
```

```
    n = len(data)
```

```
xbar = np.mean(data)
s = np.std(data, ddof=1)
alpha = 1 - confidence
t_crit = t.ppf(1 - alpha/2, df=n-1)
me = t_crit * (s / np.sqrt(n))
return xbar - me, xbar + me
```

```
# -----
# 4. CONFIDENCE INTERVAL — PROPORTION
# -----
```

```
def ci_proportion(x, n, confidence=0.95):
    p_hat = x / n
    alpha = 1 - confidence
    z = norm.ppf(1 - alpha/2)
    me = z * np.sqrt(p_hat * (1 - p_hat) / n)
    return p_hat - me, p_hat + me
```

```
# -----
# 5. CONFIDENCE INTERVAL — VARIANCE
# -----
```

```
def ci_variance(data, confidence=0.95):
    n = len(data)
```

```

s2 = np.var(data, ddof=1)
alpha = 1 - confidence
chi_low = chi2.ppf(1 - alpha/2, df=n-1)
chi_high = chi2.ppf(alpha/2, df=n-1)
lower = (n - 1) * s2 / chi_low
upper = (n - 1) * s2 / chi_high
return lower, upper

```

```

# -----

```

```

# 6. MAXIMUM LIKELIHOOD ESTIMATION (MLE)

```

```

# Example: MLE for normal distribution mean  $\mu$  and variance  $\sigma^2$ 

```

```

# -----

```

```

def mle_normal(data):
    mu_mle = np.mean(data)
    sigma2_mle = np.mean((data - mu_mle)**2)
    return mu_mle, sigma2_mle

```

```

# -----

```

```

# 7. PROPERTIES OF ESTIMATORS — BIAS, VARIANCE, MSE

```

```

# -----

```

```

def estimator_properties(estimator_values, true_value):
    estimator_values = np.array(estimator_values)

```



```
bias = np.mean(estimator_values) - true_value
variance = np.var(estimator_values, ddof=1)
mse = variance + bias**2
return bias, variance, mse
```

```
# -----
```

```
# 8. SIMULATION — Compare two estimators of variance
```

```
# -----
```

```
def simulate_estimators(sample_size=30, iterations=1000):
```

```
    biased_estimates = []
```

```
    unbiased_estimates = []
```

```
    for _ in range(iterations):
```

```
        data = np.random.normal(loc=50, scale=10, size=sample_size)
```

```
        # biased: divide by n
```

```
        biased_est = np.mean((data - np.mean(data))**2)
```

```
        # unbiased: divide by n - 1
```

```
        unbiased_est = np.var(data, ddof=1)
```

```
        biased_estimates.append(biased_est)
```

```
        unbiased_estimates.append(unbiased_est)
```

```

true_sigma2 = 10**2

biased_properties = estimator_properties(biased_estimates, true_sigma2)
unbiased_properties = estimator_properties(unbiased_estimates, true_sigma2)


return biased_properties, unbiased_properties


# -----
# EXAMPLE RUN
# -----


data = [12, 14, 16, 18, 20, 22]


print("Data:", data)
print("Point Estimates (mean, sd):", point_estimates(data))


print("\nCI Mean ( $\sigma$  known = 3):", ci_mean_sigma_known(np.mean(data), sigma=3,
n=len(data)))

print("CI Mean ( $\sigma$  unknown):", ci_mean_sigma_unknown(data))


print("\nCI for Proportion (x=60, n=100):", ci_proportion(60, 100))


print("\nCI for Variance:", ci_variance(data))


print("\nMLE for Normal ( $\mu, \sigma^2$ ):", mle_normal(data))

```

```
print("\nEstimator Simulation (Bias, Variance, MSE):")
biased, unbiased = simulate_estimators()
print("Biased Variance Estimator:", biased)
print("Unbiased Variance Estimator:", unbiased)
```

8. PYTHON CODE — ALL CONFIDENCE-INTERVAL GRAPHS

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm, t

# -----
# 1. Normal Distribution with 95% Confidence Interval
# -----

x = np.linspace(-4, 4, 500)
y = norm.pdf(x)

z_critical = 1.96
lower = -z_critical
upper = z_critical

plt.figure(figsize=(8,4))
plt.plot(x, y, linewidth=2)

# Shade the CI region
plt.fill_between(x, y, where=(x>=lower) & (x<=upper), alpha=0.3)
```

```
plt.axvline(lower, linestyle="--")
plt.axvline(upper, linestyle="--")
plt.title("95% Confidence Interval on Standard Normal")
plt.xlabel("Z-value")
plt.ylabel("Density")
plt.show()
```

```
# -----
```

```
# 2. t-Distribution vs Normal Distribution
```

```
# -----
```

```
x = np.linspace(-4, 4, 500)
```

```
plt.figure(figsize=(8,4))
plt.plot(x, norm.pdf(x), label="Normal Distribution")
plt.plot(x, t.pdf(x, df=5), label="t-Distribution (df=5)")
plt.title("Normal vs t-Distribution")
plt.xlabel("Value")
plt.ylabel("Density")
plt.legend()
plt.show()
```

```
# -----
```

```
# 3. CI Width Shrinks as Sample Size Increases
```

```
# -----
```

```
n_values = np.arange(5, 105, 5)
```

```
sigma = 10
```

```
z = norm.ppf(0.975)
```

```
ci_width = 2 * z * (sigma / np.sqrt(n_values))
```

```
plt.figure(figsize=(8,4))
```

```
plt.plot(n_values, ci_width, linewidth=2)
```

```
plt.title("CI Width vs Sample Size (n)")
```

```
plt.xlabel("Sample Size")
```

```
plt.ylabel("Confidence Interval Width")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# 4. Sampling Distribution of the Sample Mean
```

```
# -----
```

```
mu = 50
```

```
sigma = 10
```

```
samples = [np.mean(np.random.normal(mu, sigma, 30)) for _ in range(2000)]
```

```
plt.figure(figsize=(8,4))
```

```
plt.hist(samples, bins=30, density=True)
```

```
plt.title("Sampling Distribution of Sample Mean (n=30)")
```

```
plt.xlabel("Sample Mean")
```

```
plt.ylabel("Density")
```

```
plt.show()
```

```
# -----  
  
# 5. Estimator Bias & Variance Illustration  
  
# -----  
  
theta_true = 10  
  
  
# biased estimator: shifted mean, low variance  
estimator_A = np.random.normal(theta_true + 2, 1, 2000)  
  
  
# unbiased but high variance  
estimator_B = np.random.normal(theta_true, 4, 2000)  
  
  
plt.figure(figsize=(8,4))  
plt.hist(estimator_A, bins=30, alpha=0.5, density=True, label="Estimator A (Biased, Low Var)")  
plt.hist(estimator_B, bins=30, alpha=0.5, density=True, label="Estimator B (Unbiased, High  
Var)")  
  
  
plt.axvline(theta_true, linestyle="--", label="True  $\theta$ ")  
  
  
plt.title("Estimator Variance and Bias Comparison")  
plt.xlabel("Estimator Value")  
plt.ylabel("Density")  
plt.legend()  
plt.show()  
  
# -----
```

```
# 6. Likelihood Function for MLE (Normal Mean)
```

```
# -----
```

```
data = np.array([48, 50, 51, 49, 52])
```

```
theta_vals = np.linspace(45, 55, 300)
```

```
def likelihood(theta):
```

```
    return np.prod(norm.pdf(data, loc=theta, scale=1.5))
```

```
L_vals = np.array([likelihood(t) for t in theta_vals])
```

```
plt.figure(figsize=(8,4))
```

```
plt.plot(theta_vals, L_vals, linewidth=2)
```

```
plt.title("Likelihood Function for Mean (MLE)")
```

```
plt.xlabel(" $\theta$ ")
```

```
plt.ylabel("Likelihood")
```

```
plt.show()
```

```
9.point estimation vs. interval estimation
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Example data
```

```
x = np.arange(1, 11)
```

```
true_values = np.ones_like(x) * 10      # True parameter value = 10
```

```
point_estimates = 10 + np.random.randn(10) # Random estimates around true value
```

```
ci_lower = point_estimates - np.random.uniform(0.5, 2, 10)
```

```
ci_upper = point_estimates + np.random.uniform(0.5, 2, 10)

# Plot

plt.figure(figsize=(8,5))

plt.plot(x, point_estimates, marker='o', label='Point Estimate')
plt.vlines(x, ci_lower, ci_upper, label='Confidence Interval')
plt.axhline(10, linestyle='--', label='True Value')

plt.xlabel("Sample Index")
plt.ylabel("Estimate")
plt.title("Point Estimation vs Interval Estimation")
plt.legend()
plt.tight_layout()
plt.show()
```